

Project #1

Multi-Point Maze Navigation: Capture the Flags

Capstone Design [202601-ICE/CSE4020]

Professor: Dr.Oybek Eraliev

Team Assignment (6–8 members)



1. Overview

In this project, your team will program a TurtleBot3 to autonomously navigate through a maze and collect **5 flags** placed at random locations. The robot must detect obstacles in real time, plan collision-free paths, and — most importantly — decide which flag to pursue next by always choosing the **nearest reachable flag** relative to its current position.

The maze layout will be revealed by the university staff in our campus. Further details are announced gradually on our Discord. You may not hardcode paths, waypoints, or any maze-specific knowledge into your code. The robot must sense its environment and react dynamically.

This project is a significant step up and it brings together simulation, physical robotics, sensor processing, path planning, and algorithmic decision-making into a single integrated system. Your team will need to divide responsibilities, test thoroughly, and ultimately demonstrate your solution on a physical TurtleBot3.

2. Technical Background

This section outlines the key tools and concepts your team will need. It is not a step-by-step tutorial — you are expected to research, experiment, and learn the details on your own.

2.1 TurtleBot3 and Gazebo Simulation

Your development and testing environment could be the **Gazebo** simulator with a TurtleBot3 model (Burger) or you could experiment with the physical maze in our campus earlier. Gazebo provides a physics engine, sensor simulation (including LiDAR), and the ability to load custom world files.

2.2 Sensor Data and Obstacle Detection

The TurtleBot3 is equipped with a **360° LiDAR sensor** that publishes range data on the `/scan` topic as `sensor_msgs/LaserScan` messages. Each message contains an array of distance measurements covering the full rotation around the robot. This is your primary tool for detecting walls and obstacles. How you process this data — whether by examining specific angular sectors, computing minimum distances in regions of interest, or feeding it into a more sophisticated algorithm — is a design decision for your team.

2.3 Localization and Odometry

To compute distances to flags, your robot needs to know its own position within the maze. The `/odom` topic provides odometry data (`nav_msgs/Odometry`) based on wheel encoders, which gives you an estimate of the robot's position and orientation. Be aware that odometry drifts over time. For more accurate localization, you may explore the **AMCL** (Adaptive Monte Carlo Localization) package, which uses LiDAR data and a known map to correct the robot's estimated pose. Whether you use raw odometry, AMCL, or another approach is up to your team.

2.4 Navigation and Path Planning

Getting the robot from its current position to a target flag without hitting walls is the core engineering challenge. The ROS **Navigation Stack** (`move_base`) provides a framework for autonomous navigation that integrates global path planning, local obstacle avoidance, and costmaps. However, using `move_base` is **not mandatory** — teams are free to implement their own navigation logic from scratch using velocity commands and LiDAR data, to use a wall-following strategy, to implement a bug algorithm, or to take any other approach. What matters is that the robot reaches each flag without collisions.

2.5 Flag Representation in Simulation

In the Gazebo simulation, the 5 flags will be represented as visual markers placed at specific coordinates within the maze. The coordinates of all flags will be provided to your node at startup (the exact delivery method — such as a YAML parameter file, a ROS topic, or a configuration file — will be announced along with the maze). A flag is considered **captured** when the robot comes within a defined **capture radius** (e.g., 0.3 meters) of the flag's position. Once captured, the flag should be removed from the robot's active target list and, if you choose, visually despawned from the simulation.

3. Project Requirements

Your team must deliver a working system that satisfies all of the following requirements.

3.1 Autonomous Navigation

The robot must navigate the maze entirely on its own. No manual control via keyboard teleoperation or joystick is permitted during a run. The robot must start at a designated spawn position and proceed to collect all flags without human intervention.

3.2 Obstacle Avoidance

The robot must not collide with maze walls or any other obstacles during navigation. Your system should continuously monitor LiDAR data and adjust the robot's trajectory to prevent collisions. The specific avoidance strategy is entirely up to your team.

3.3 Nearest-Flag Selection

At each decision point (after capturing a flag or at the start), the robot must compute distances to all remaining flags and select the nearest one as the next target. This selection logic must be visible in your code and verifiable from your recorded output.

3.4 Flag Capture

When the robot is within the defined capture radius of a flag, it must recognize the capture, log it (e.g., print to the terminal or publish to a topic), and remove the flag from its active target list. The robot must successfully capture all 5 flags to complete the run.

Hints and Guidance

- Start simple. Get your robot to navigate to a single known coordinate in an empty world before adding maze walls, multiple flags, and selection logic.
- Build your test mazes early in Gazebo using simple box models or the Building Editor. Do not wait for coordinations - try to experiment yourself first.
- Print or log the distances to all flags at each decision point. This makes debugging much easier and also helps with the report.
- Consider using RViz to visualize the robot's position, LiDAR data, costmaps, and flag locations simultaneously. It is invaluable for debugging navigation issues.
- Physical robot behavior often differs from simulation due to wheel slippage, sensor noise, and latency. Allocate significant time for real-world testing and parameter tuning.
- Divide the work within your team clearly: navigation, obstacle avoidance, flag management, simulation setup, physical testing, and report writing are all distinct workstreams.

4. Grading Rubric

Your submission will be evaluated according to the following criteria. The total score is 100 points.

Criterion	Points	Details
Autonomous navigation <i>no manual control</i>	20	The robot navigates the maze from start to all flag locations without any human intervention or teleoperation.
Obstacle avoidance	20	The robot avoids all walls and obstacles throughout the run. No collisions observed in simulation or physical demo.
Successful flag capture <i>all 5 flags required</i>	50	All 5 flags are captured (<i>10 pts per flag</i>).
Written report <i>see Section 5 for details</i>	10	Clear description of the approach, team member contributions, architecture diagram, challenges faced, and results analysis.

5. Submission Instructions

Step 1:

Your team must submit the following items through the specified platforms.

1. **Full ROS package** — Submit your complete ROS package as a private repository on the **GitHub** of the course – IUT-SCS (**ONLY ONE REPOSITORY PER TEAM**) – before a deadline. The repository must include all source code, launch files, configuration files, and a [README.md](#) with clear instructions on how to build and run your solution. The package should be self-contained: a reviewer should be able to clone the repository, build with `catkin_make`, and launch your solution.
2. **Written report** — Submit a PDF report (3–6 pages) to the **e-Class** before the deadline. The report must include the following sections:
 - **Approach and Architecture** — Describe your navigation strategy, obstacle avoidance method, and flag selection logic. Include a high-level architecture diagram showing your ROS nodes, topics, and services.
 - **Team Contributions** — Clearly list each team member and their specific responsibilities and contributions to the project.
 - **Challenges and Solutions** — Discuss the key technical challenges your team encountered and how you addressed them.
 - **Results** — Summarize the outcome of your runs. Include the order in which flags were captured and the total time if measured.

Late submissions will be penalized!

-10 pts. for a late submission.

allowed number of days after the deadline: **2 (including that day)**.

Step 2:

Your team must pass the process of testing your TurtleBot3 on our campus.

3. **Running in maze** — Each team gets its own slot to run the robot in the maze, and it's requested to submit its entire work as a team (**ALL MEMBERS MUST BE PRESENT DUE TO INDIVIDUAL GRADING**). Registering teams is done by a **FIFO method** according to the timing of submission of both the Full ROS Package and the written report.
4. **Double recording** — Two devices must be used to record a team's robot running process. One for recording the maze and the robot in it. Another is for recording the laptop that is used to run the robot. *(What devices to choose and what tools to use is entirely up to you.)*

5. **Submission of videos** – Two recorded videos that are stated above (in “4.”) must be uploaded to a YouTube channel, and the link must be shared with the course administration – either the EA or Professor.

Although there’s no deadline for “5.”, it’s better to upload your videos and share necessary links right after your successful run!

6. Important Notes

1. This is a **team assignment**. All team members are expected to contribute meaningfully. The written report must detail each member’s contributions. Teams with members who did not contribute may have individual grades adjusted. *(Each member is recommended to do gradual pushes to a team repo so it’s possible to see what has been done by whom afterwards.)*
2. Teams are responsible for the condition of their TurtleBot3 hardware. Handle the robots with care, keep batteries charged *(be prepared earlier)*, and report any damage immediately in your team space on Discord.
3. Sharing code between teams is not permitted. Each team’s solution must be developed independently. Reusing open-source ROS packages (such as the Navigation Stack) is allowed and encouraged, but your integration and decision logic must be original work.
4. The physical demonstration session will be scheduled separately. Details, including the date and location, will be announced on the **Discord server**.
5. Start early and divide the work. This project involves simulation setup, algorithm development, ROS integration, physical testing, and report writing — there is enough work for every team member.

Good luck! If you have any questions, reach out to the Education Assistant of this course.